ARM® Cortex®-M3 Processor Toolchain Reference

for Ezairo® 7100

M-20803-005
September 2014



ON Semiconductor®

© SCILLC, 2014
Previous Edition © 2013
"All Rights Reserved"

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction 4
    1.1 Purpose 4
    1.2 Intended Audience 4
    1.3 Conventions 4
    1.4 Manual Organization 4
    1.5 Further Reading 5
2. Overview of the Tools 6
    2.1 GCC 6
    2.2 Binutils (GNU binary utilities) 6
    2.3 GNU Make 7
3. Compiling C Source Files 8
    3.1 Recommended Compiler Options 8
    3.2 Using the Compiler from the Command Line 9
4. Linking Executables. 11
    4.1 Recommended Linker Options 11
    4.2 Using the Linker from the Command Line 12
5. Building Libraries 13
    5.1 Default Librarian Options 13
6. Defining Interrupt Handlers 14
    6.1 Overriding an Interrupt Handler 14
7. Converting ARM Executable Files to CFX Assembly Files 16
    7.1 Setting Up and Running the Tool 16
    7.2 Using ELF2CFS from the Command Line 16
    7.3 ELF2CFS Output 16

CHAPTER 1
Introduction

1.1 Purpose

This *ARM® Cortex®-M3 Processor Toolchain Reference for Ezairo® 7100* provides information for building programs to run on an ARM Cortex-M3 processor that is part of Ezairo 7100. It describes the compiling and linking of C programs using GNU tools. It also describes how to build libraries, define interrupt handlers, and merge the executable files with programs that run on the CFX processor. The build process can be performed from a command line or through the Integrated Development Environment (IDE).

The ARM Cortex-M3 processor toolchain is derived from Sourcery G++ developed by CodeSourcery. This manual only deals with the particular information that you need to use the toolchain for Ezairo 7100. For the standard information about Sourcery G++, see Section 1.5, “Further Reading” on page 5.

1.2 Intended Audience

This manual was written for readers who know the GNU toolchain and have some familiarity with the C language. For more information on concepts such as the C preprocessor, C-style comments, C expression operators, C-style numeric literals, and C string escape sequences, see any of the standard works on the subject, such as *The C Programming Language* by Kernighan and Ritchie.[1]

1.3 Conventions

In general, numbers are presented in decimal notation. In cases where hexadecimal notation is more convenient, these numbers are identified by the prefix “0x”. For example, the decimal number 123456 can also be represented as 0x1E240.

The following special fonts are used in this manual to signify particular types of information:

`monospace font`	Commands and their options, file and path names, error messages, code samples and code snippets.
`mono bold`	A placeholder that represents where you would specify the appropriate information. For example, you would replace **`filename`** with the actual name of the file.
bold	Used for menu names and menu items.
`<EDK_HOME>`	The folder where you installed the Evaluation and Development Kit software. See the *Ezairo 7100 Installation Guide* for more information.

1.4 Manual Organization

The *ARM Cortex-M3 Processor Toolchain Reference* contains the following chapters:

- **Chapter 1: Introduction**, describes the purpose of this manual and how the book is organized, and gives a list of suggested reading for more information.
- **Chapter 2: Overview of the Tools**, provides a description of the main components of the GNU toolchain for the ARM Cortex-M3 processor.
- **Chapter 3: Compiling C Source Files**, recommends GCC compiler settings.

1. Kernighan, Brian W., and Ritchie, Dennis M. The C Programming Language. 2nd ed. Prentice Hall, 1988.

- **Chapter 4: Linking Executables**, recommends GCC linker settings.
- **Chapter 5: Building Libraries**, describes the tool used to build libraries.
- **Chapter 6: Defining Interrupt Handlers**, gives an overview of using the interrupt handlers.
- **Chapter 7: Converting ARM Executable Files to CFX Assembly Files**, describes the tool ELF2CFS and how to use it.

1.5 Further Reading

Available from Eclipse User Assistance when you start the IDE:

- *CFX DSP Architecture Manual*
- *SK4 ELF Toolchain Reference for Ezairo 7100*
- *Eclipse Workbench User Guide*
- *Evaluation and Development Board Manual for Ezairo 7100*
- *Ezairo 7100 Installation Guide*
- *Firmware Reference Manual for Ezairo 7100*
- *Integrated Development Environment User's Guide for Ezairo 7100*
- *Introduction to Ezairo 7100 Programming*
- *Hardware Reference Manual for Ezairo 7100*
- *HEAR Configurable Accelerator Reference Manual*

You can find the redistributable G++ manuals as part of the EDK installation in **<EDK_HOME>***arm_tools**share**doc**arm-arm-none-eabi**pdf*, where **<EDK_HOME>** is the folder with this Evaluation and Development Kit software installed. See the *Ezairo 7100 Installation Guide* for more information about finding **<EDK_HOME>**.

- *Sourcery G++ Lite Getting Started*
- *Using the GNU Compiler Collection*
- *Using as: The GNU Assembler*
- *The GNU Linker Manual*
- *The GNU Binary Utilities*

Other manuals that are publicly available:

- *GNU Make Manual* (available online)
- Kernighan, Brian W., and Ritchie, Dennis M. *The C Programming Language*. 2nd ed. (Prentice Hall, 1988)

CHAPTER 2
Overview of the Tools

Ezairo 7100 uses CodeSourcery's[2] Sourcery G++ as the professional C/C++ development tool suite for the ARM Cortex-M3 processor. This development suite includes GCC, binutils, and GNU Make, which are required to compile, run and debug code. The following sections focus on these components of the suite.

For more information, see the *Sourcery G++ Lite Getting Started* manual.

2.1 GCC

The GCC compiler is the primary component of the GNU toolchain. Originally designed as a compiler for C, it now supports C++, Ada, Fortran, Objective C, and even Java. For the purposes of this manual, we will consider only the C/C++ functionality of the toolchain.

For more information, see *Using the GNU Compiler Collection* manual.

2.2 BINUTILS (GNU BINARY UTILITIES)

GCC depends on many external tools, most of which are binary tools collectively called binutils. You can use them to create and manage binary programs, object files, libraries, profile data, and assembly source code. The tools we are providing with this EDK are:

addr2line	Convert addresses into file names and line numbers
ar	Create, modify, and extract from archives
as	The GNU assembler. For more information, see *Using as: The GNU Assembler* manual.
ld	The GNU linker. For more information, see *The GNU Linker Manual*.
nm	List symbols from object files
objcopy	Copy and translate object files
objdump	Display information from object files
ranlib	Generate index to archive contents
readelf	Display the contents of ELF format files.
strip	Discard symbols

For more information on binutils and other tools included in it, see *The GNU Binary Utilities* manual.

2. For the latest version of Sourcery G++, contact Mentor Graphics (which bought CodeSourcery) about Sourcery CodeBench.

2.3 GNU MAKE

We use GNU Make to control the build process. Whenever the `make` command is called, a Makefile contained in the directory is interpreted and executed to produce target outputs. For more information, see the *GNU Make Manual*.

CHAPTER 3
Compiling C Source Files

This chapter discusses the recommended compiler options for developing an ARM Cortex-M3 processor project for Ezairo 7100.

3.1 Recommended Compiler Options

The following instructions assume you are working in the IDE. For more information about using the IDE, see the *Integrated Development Environment User's Guide for Ezairo 7100*.

1. In the C/C++ perspective, go to the **Project** menu > **Properties** menu item > expand **C/C++ Build** > choose **Settings** > **Tool Settings** tab > select **GCC Compiler**. You see a window similar to Figure 1.

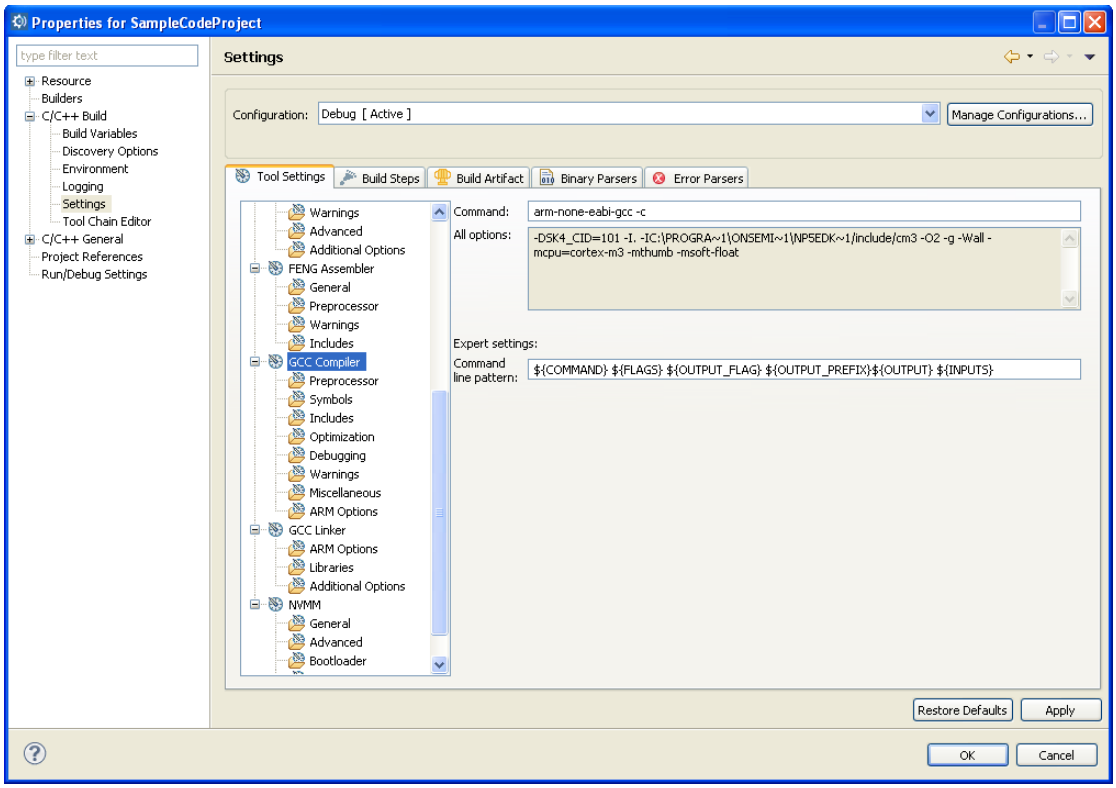


Figure 1. Finding the GCC Compiler Options

- Set **Command** to `arm-none-eabi-gcc -c`. This sets eabi as the default option for GCC.
- Set **Command line pattern** to `${COMMAND} ${FLAGS} ${OUTPUT_FLAG} ${OUTPUT_PREFIX}${OUTPUT} ${INPUTS}`.

2. Under **GCC Compiler**, select **Preprocessor** options. The included files and #defines inside the code are substituted with the actual source code and defined values by the preprocessor before the project is compiled.
   - Ensure that **Do not search system directories (-nostdinc)** is unchecked.
   - Ensure that **Preprocess only (-E)** is unchecked.

3. Under **GCC Compiler**, select **Symbols**, where you can declare the variable definitions for the project.
   - Add `SK4_CID=${chip.id}` in the **Defined Symbols (-D)** field.
   - Leave the **Undefined Symbols (-U)** field blank.

4. Under **GCC Compiler**, select **Includes** to add the directories that contain all the referenced files. In the **Included paths(-I)** field, add:

```
.
${edk.include.cm3}
```

5. Under **GCC Compiler**, select **Optimization** to set optimization options to improve the performance.
   - Set the **Optimization Level** to `Optimize more (-O2)`. This level increases the optimization, while balancing it with fast compilation and debugging.
   - Leave the **Other optimization flags** field blank.

6. Under **GCC Compiler**, select **Debugging** to set options that determine the information the compiler stores for debugging.
   - Set `Default (-g)` for the **Debug Level**. This is a standard debug level option that inserts debug information such as line numbers within the code.
   - Leave the **Other debugging flags** field blank.
   - Ensure that **Generate prof information (-p)** is unchecked.
   - Ensure that **Generate gprof information (-pg)** is unchecked.

7. Under **GCC Compiler**, select **Warnings**.
   - Ensure that only **All warnings (-Wall)** is checked. This shows you all the warnings that are encountered during compilation.

8. Under **GCC Compiler**, select **Miscellaneous**.
   - Ensure that the **other flags** field is blank.
   - Ensure that all other options are unchecked.

9. Under **GCC Compiler**, select **ARM Options**. You can change the settings for the hardware in this category.
   - Set **CPU Type(-mcpu=)** to `cortex-m3`.
   - Set **Instruction Set (-m)** to `thumb`.
   - Ensure that **Software floating point emulation (-msoft-float)** is checked. The program then emulates floating point in software because the hardware does not have a floating point processor.

3.2 Using the Compiler from the Command Line

The following instruction shows the recommended compiler options to set from the command line.

Command: `arm-none-eabi-gcc -c -DSK4_CID=102 -I. -IC:\PROGRA~1\ONSEMI~1\EZAIRO~1/include/cm3 -O2 -g -Wall -mcpu=cortex-m3 -mthumb -msoft-float -o "Cortex-M3-src/stack.o" "../Cortex-M3-src/stack.c"`

-D`SK4_CID=102`
Defines the processor chip ID.

-I . and -I`C:\PROGRA~1\ONSEMI~1\EZAIRO~1/include/cm3`
Includes the directory containing the libraries and files used in the project

*-O*2
: Increases the optimization, while balancing it with fast compilation and debugging.

-g
: Turns on debugging information.

*-W*all
: Enables all construction warnings.

-mcpu=cortex-m3
: Defines the processor to use.

*-m*thumb
: Uses the Thumb instruction set.

*-m*soft-float
: Disables the generation of floating point instructions from the hardware, forcing it to use software floating point instructions instead.

-o "Cortex-M3-src/stack.o"
: Defines the name of the output file.

"../Cortex-M3-src/stack.c"
: The file being compiled.

For more information about command line compiler options, see the *Using the GNU Compiler Collection* manual.

CHAPTER 4
Linking Executables

This chapter discusses the recommended GCC linker settings.

4.1 Recommended Linker Options

The following instructions assume you are working in the IDE. For more information about using the IDE, see the *Integrated Development Environment User's Guide for Ezairo 7100*.

1. In the C/C++ perspective, go to the **Project** menu > **Properties** menu item > expand **C/C++ Build** > choose **Settings** > **Tool Settings** tab > select **GCC Linker**. You see a window similar to Figure 2.

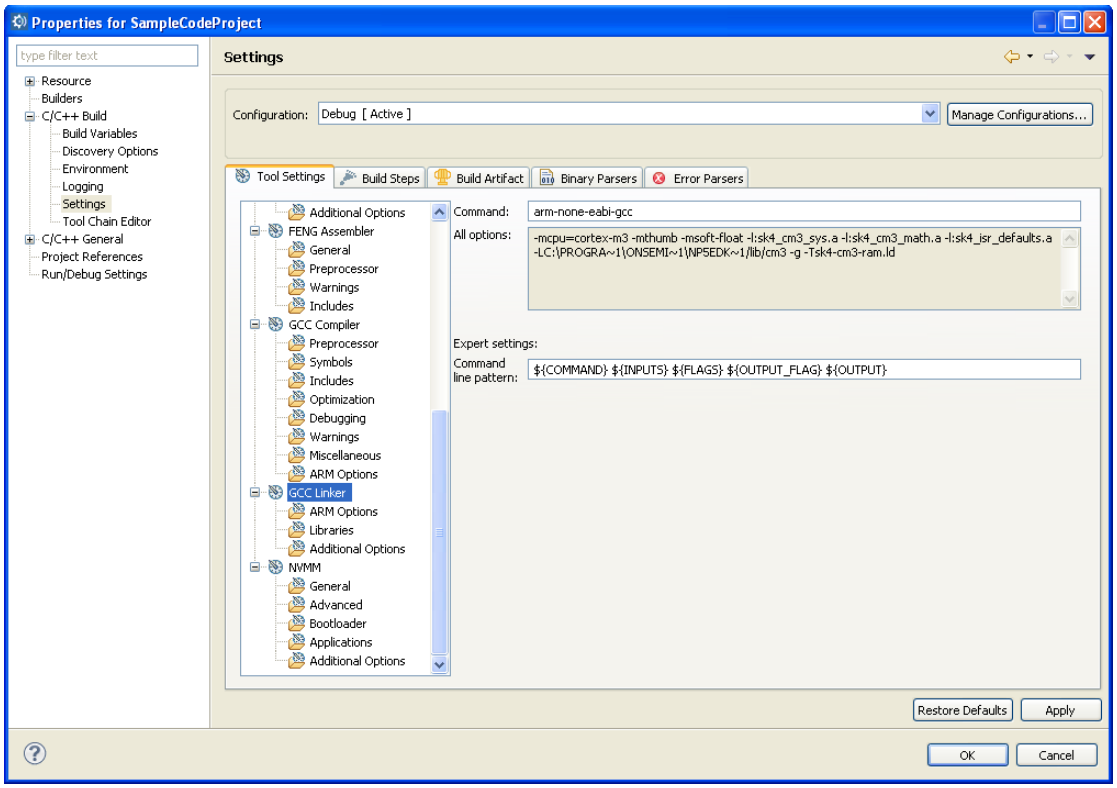


Figure 2. Finding the GCC Linker Options

- Set **Command** to `arm-none-eabi-gcc`. This sets eabi as the default option for GCC.
- Set **Command line pattern** to `${COMMAND} ${INPUTS} ${FLAGS} ${OUTPUT_FLAG} ${OUTPUT}`.

2. Under **GCC Linker**, use **ARM Options** to set the options for the hardware. We recommend the following settings:
   - Set **CPU Type(-mcpu=)** to `cortex-m3`.
   - Set **Instruction Set** to `thumb`.
   - Ensure that **Software floating point emulation (-msoft-float)** is checked. The program then emulates floating point in software because the hardware does not have a floating point processor.

3. Under **GCC Linker**, select **Libraries**. The recommended settings for **Libraries** in the top panel are:
   - `sk4_cm3_sys.a`
   - `sk4_cm3_math.a`
   - `sk4_isr_defaults.a`

   The recommended setting for **Libraries** in the bottom panel is `${edk.lib.cm3}`.

4. Under **GCC Linker**, select **Additional Options**.
   - Set **Additional Options** to `-g -Tsk4-cm3-ram.ld`. The option `-g` adds debug information while linking. The option `-T` maps memory from a file that is in **<EDK_HOME>***lib\cm3\sk4-cm3-ram.ld* in this scenario.

4.2 Using the Linker from the Command Line

The following instructions show the recommended linker options to set from the command line.

Command: `arm-none-eabi-gcc ./Cortex-M3-src/initialize.o ./Cortex-M3-src/main.o ./Cortex-M3-src/radio.o ./Cortex-M3-src/stack.o -mcpu=cortex-m3 -mthumb -msoft-float -l:sk4_cm3_sys.a -l:sk4_cm3_math.a -l:sk4_isr_defaults.a -LC:\PROGRA~1\ONSEMI~1\EZAIRO~1/lib/cm3 -g -Tsk4-cm3-ram.ld -o "Wireless.elf"`

`./Cortex-M3-src/initialize.o ./Cortex-M3-src/main.o ./Cortex-M3-src/radio.o ./Cortex-M3-src/stack.o:`
Defining all the output files from the project that will be linked together.

-mcpu=`cortex-m3`
Defines the processor to use.

-m`thumb`
Uses the Thumb instruction set.

-m`soft-float`
Disables the generation of floating point instructions from the hardware, forcing it to use software floating point instructions instead.

-l:`sk4_cm3_sys.a` *-l:*`sk4_cm3_math.a` *-l:*`sk4_isr_defaults.a`
Identifies the libraries that need to be linked.

-L`C:\PROGRA~1\ONSEMI~1\EZAIRO~1/lib/cm3`
Defines the path to search for libraries and `ld` scripts.

-g
Turns on debugging information.

-T`sk4-cm3-ram.ld`
Reads link commands from the specified file.

-o `"Wireless.elf"`
Defines the name of the output executable file.

For more information about command line linker options, see the *The GNU Linker Manual*.

CHAPTER 5
Building Libraries

This chapter describes the tool used to build libraries for Ezairo 7100.

5.1 Default Librarian Options

Ezairo 7100 uses a standard implementation of the GNU archiver, `ar`, to build libraries. This tool is part of the binutils package. For more information, see *The GNU Binary Utilities* manual.

CHAPTER 6
Defining Interrupt Handlers

Interrupts are fired at various instances during the execution of the program, and handlers are used to define actions that occur when that happens. By default, the handlers declared in **<EDK_HOME>***source**cm3**isr_defaults**sk4_isr_defaults.h* are set to do nothing on interrupt. This chapter discusses how to override these interrupt handlers with more useful implementations.

6.1 Overriding an Interrupt Handler

The handlers are declared in **<EDK_HOME>***source**cm3**isr_defaults**sk4_isr_defaults.h* but implemented in other project files, as defined by `extern` directive. By default, these handlers are weak-linked to a function as defined in **<EDK_HOME>***source**cm3**isr_defaults**handlers.c*. The weak link means that if other definitions of the handler are found, they are given preference. The handler names correspond to the interrupt names, but for a clear relationship between the handler and the interrupt, see **<EDK_HOME>***source**cm3**isr_defaults**vector_table.c*.

To override the handler, first link the project to the *isr_default* library to import it and get definitions of all the interrupt handlers. During the initialization of the program, you must enable the interrupts you are going to use. The `enum` value used for enabling interrupts is defined in **<EDK_HOME>***include**cm3**sk4_vectors.h*. To perform an action during the enabled interrupt, you can override the interrupt handler in the source code to change its default behavior. This does not affect the default behavior of the other interrupts.

The following is an example of the `FIFO_0_IRQHandler` interrupt handler from the sample code:

- The definition in **<EDK_HOME>***source**cm3**isr_defaults**sk4_isr_defaults.h* is:

```
extern void FIFO_0_IRQHandler(void);
```

- The definition in **<EDK_HOME>***source**cm3**isr_defaults**handlers.c* is:

```
void FIFO_0_IRQHandler(void)   __attribute__((weak,
alias("Default_Handler")));
```

- The definition in **<EDK_HOME>***source**cm3**isr_defaults**vector_table.c* is:

```
{ FIFO_0_IRQHandler },           /* 16: FIFO interrupt 0                  */
```

- Steps to override the `FIFO_0_IRQHandler` interrupt handler:
  a. In the initialization file, enable the FIFO interrupt 0:

```
NVIC_EnableIRQ(FIFO_0_IRQn);
```

  b. Un-mask interrupts:

```
__set_PRIMASK(PRIMASK_ENABLE_INTERRUPTS);
```

  c. In the source code, override the interrupt handler:

```
/*
 * Function      : void FIFO_0_IRQHandler(void)
 *
 * Description   : Handler for the FIFO 0 interrupt.
 * Inputs        : None
 * Outputs       : None
 * Assumptions   : None
 */
void FIFO_0_IRQHandler(void)
```

```
{
// Put Fifo interrupt 0 code here
}
```

CHAPTER 7
Converting ARM Executable Files to CFX Assembly Files

If you have CFX and ARM Cortex-M3 processor source code in one project, the ARM Cortex-M3 processor code is turned into a CFX executable file to give the CFX control over all the cores in the system.

ELF2CFS is the tool that converts ARM Cortex-M3 processor executable files into CFX assembly code which can be loaded through the CFX core. This results in both cores being used and handled by separate programs. This chapter provides more information about ELF2CFS, how it can be used and its resulting output.

7.1 Setting Up and Running the Tool

ELF2CFS is a special tool and is not accessible through the build settings dialog; instead, it is defined in the *makefile.targets* and *makefile.defs* files. These files allow you to specify additional targets and by default, it adds a pattern rule for ELF2CFS. The default output file name is *CM3.cfs*.

For an example, see the Wireless Audio Passthrough sample provided with the EDK. For more information about the *makefile.** files and using the sample code, see the *Integrated Development Environment User's Guide for Ezairo 7100*. For more information about the Wireless Audio Passthrough sample, see the *Introduction to Ezairo 7100 Programming*.

7.2 Using ELF2CFS from the Command Line

To run ELF2CFS from the command line, use:

```
elf2cfs ELF-executable-input [output.cfs]
```

where:

ELF-executable-input is a *.elf* file

output.cfs is an optional file name for the output, which goes to the console if a file name is not provided

7.3 ELF2CFS Output

ELF2CFS converts an ARM Cortex-M3 processor executable file (*.elf*) to data blocks in a file format (*.cfs*) that is usable by the CFX toolchain. The regular linker (see the *SK4 ELF Toolchain Reference for Ezairo 7100* for more information) then combines this output file with the CFX code to create an executable file (*.cfe*) that can be run by the CFX.

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative